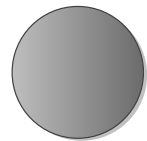
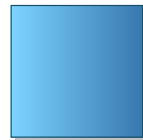


Database Management System

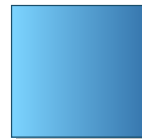


Vijay



SBI - Chennai

Passbook System
Excel Data



SBI - Coimbatore

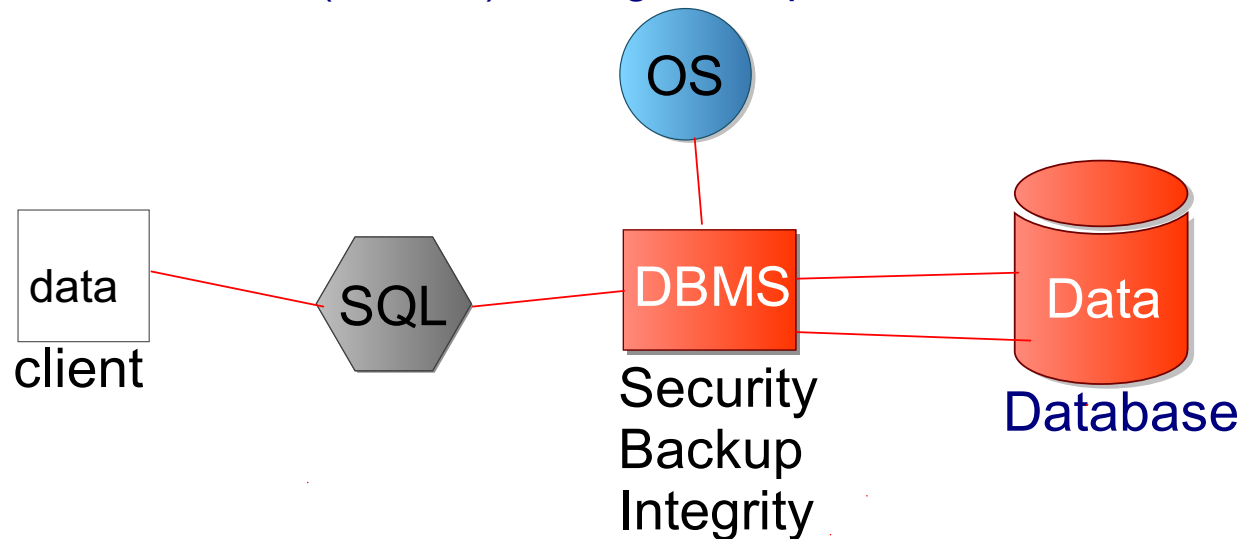
Durability
Inconsistency
Availability
Security

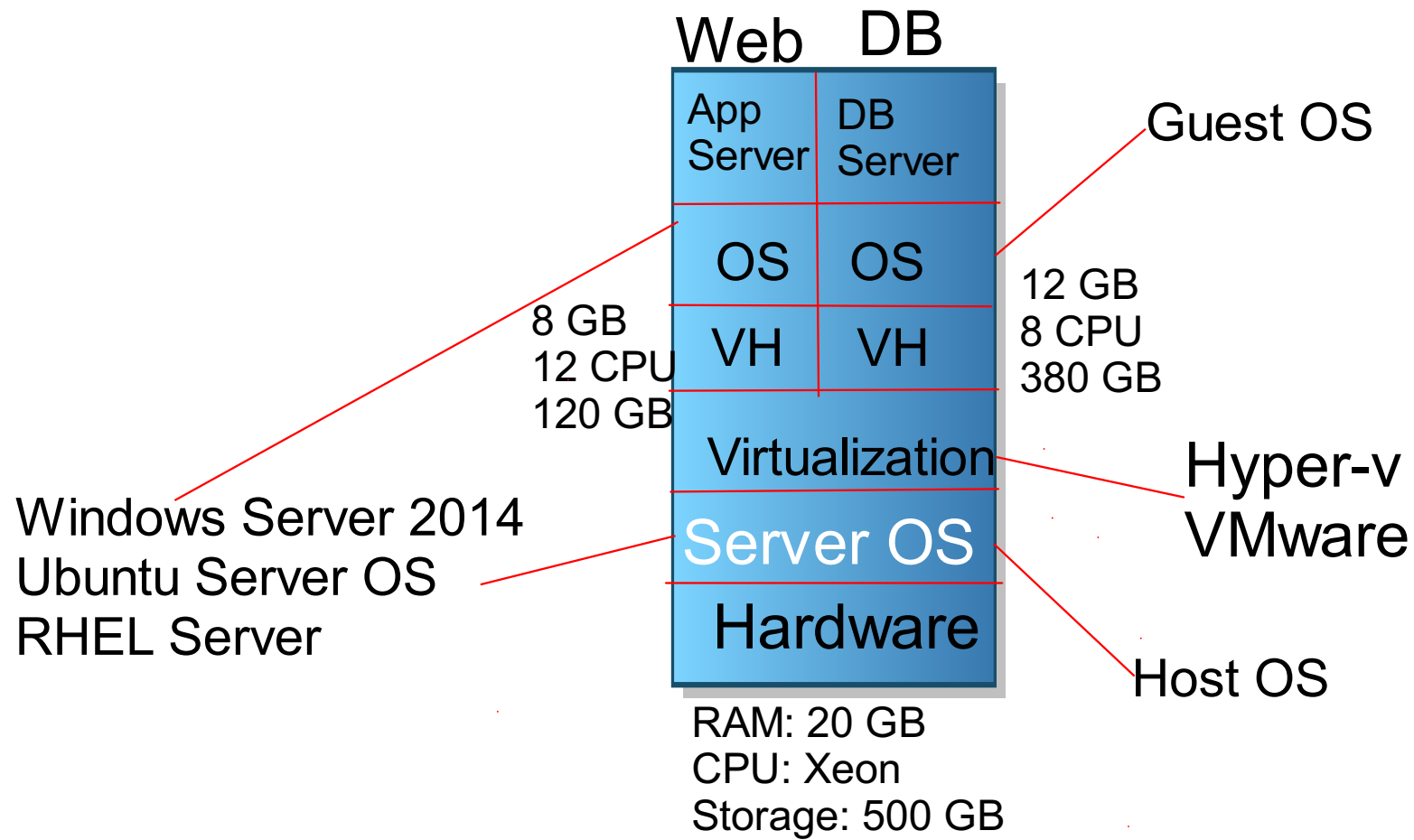
Types of Data

- Unstructured : Files in system - Harddisk, pendrives etc.,
- Semistructured : Email, JSON, XML
- Structured: Databases (Relational and NoSQL)

Types of Databases

- Structured(Relational) : MySQL, Oracle, PostgreSQL, MS SQL
- Unstructured(NoSQL) : MongoDB, Apache Cassandra





Web/App Server		Database Server		Server/Client Systems	
HTTP	80	MySQL	3306	Windows RDP	3389
HTTPS	443	MS SQL	1433	Linux SSH	22
		Oracle	1521		

App Servers

Java	Tomcat/JBoss
.NET	IIS
Python	WebSphere
Node.js	GlassFish

Structured Query Language

- DDL (create, alter, drop, ...)
- DML (insert, select, update, delete, ...)
- TCL (commit, rollback,...)
- DCL (TRANSACTION, GRANT, REVOKE)

MySQL - DBMS Tool

Connectivity:

- Hostname: localhost/127.0.0.1
- Port: 3306
- Admin Port: 33062
- username: root
- password:

a

customer_id	user_name	password
1	samantha	a1fe7c4fe98b06835c85f7d6ac410eb1

b

customer_id	name	account_number
1	Samantha	8949848
2	Gokul	8949949
3	Aslam	8949950
4	Ranjith	8949952

select a.customer_id, b.name, b.account_number from

customer_id	name	account_number
1	Samantha	8949848

customer_id	name	account_number	user_name	password
1	Samantha	8949848	samantha	a1fe7c4fe98b06835c85f7d6ac410eb1
2	Gokul	8949949	gokul	a1fe7c4fe98b06835c85f7d6ac410eb1
3	Aslam	8949950	NULL	NULL
4	Ranjith	8949952	NULL	NULL

customer_id	name	account_number	transaction_id	amount	transaction_type	transacted_on
1	Samantha	8949848	3	45678.00	2	2026-01-10 14:59:17
1	Samantha	8949848	2	345.00	1	2026-01-10 14:59:17
1	Samantha	8949848	1	2345.00	1	2026-01-10 14:59:17
2	Gokul	8949949	4	2345.00	2	2026-01-10 14:59:17
3	Aslam	8949950	NULL	NULL	NULL	NULL
4	Ranjith	8949952	NULL	NULL	NULL	NULL

transacted_on	transaction_type	amount
2026-01-10 14:59:17	1	2345.00
2026-01-10 14:59:17	1	345.00
2026-01-10 14:59:17	2	45678.00

transacted_on	transaction_type	name	amount
2026-01-10 14:59:17	1	Withdrawl	345.00
2026-01-10 14:59:17	1	Withdrawl	2345.00
2026-01-10 14:59:17	2	Deposit	45678.00

customer_id	name	account_number	transaction_id	amount	transaction_type	transacted_on
1	Samantha	8949848	1	2345.00	1	2026-01-10 14:59:17
1	Samantha	8949848	2	345.00	1	2026-01-10 14:59:17
1	Samantha	8949848	3	45678.00	2	2026-01-10 14:59:17
2	Gokul	8949949	4	2345.00	2	2026-01-10 14:59:17

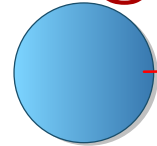
<https://tinyurl.com/bytss2025>

User Management(RBAC)

localhost:3306

root

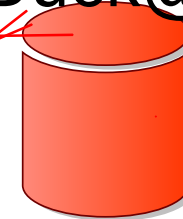
Duck@!4#



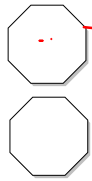
client 1

root

Duck@!4#



MySQL

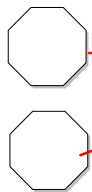


root

Duck@!4#

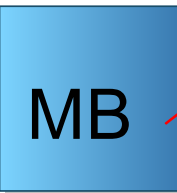


loan



root

Duck@!4#



MB

Development
team

Order Processing

```
order_id  bigint auto_increment
customer_id int
product_id text -- comma seperated values
product_name text
product_price text
product_quantity text
order_price decimal(4,2)
address text
delivery_date datetime
```

78 samantha chennai

79 trisha coimbatore

45 iPhone15 78955

47 iPhone16 97854

orders

123456	78	"45"	"iPhone15"	"78955"	78955	"1"	"chennai"	15-01-2026
123457	79	"45, 47"	"iPhone15, iPhone16"	"78955, 97854"	255764	"2, 1"		

Object Oriented Programming Concepts

SOLID Principles

Design Patterns

Coding Guidelines

Project Structure

DevOps (CI/CD)

Application Types

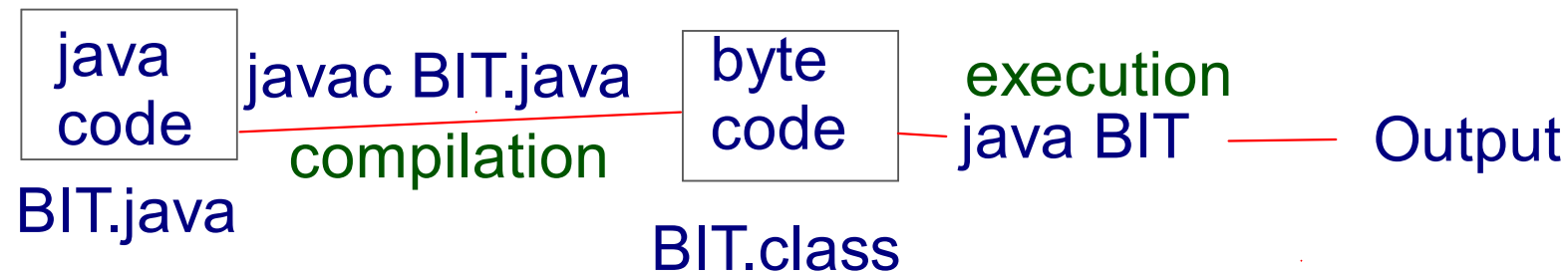
client-server architecture

Application Architectures (Monolithic, n-tier, MicroService)

Object Oriented Programming Concepts

//class is a template or blueprint

```
class BIT{  
    //Instance variable  
    int num = 10;  
    //method  
    public static void main(String[] a){  
        System.out.print("Hello! BIT");  
    }  
}
```



Members of a class : data(variables, fields, properties) and behavior(methods)

```
class Test{  
    //Instance variables  
    //Constructors  
    //Static Blocks  
    //Inner classes  
    //abstract or concrete methods  
}
```

```
static Integer id;  
String name;  
String email;  
String department;
```

Student ae367g

```
Integer id;  
String name;  
String email;  
String department;
```

Student std2 = new Student();

```
Integer id; 1  
String name; Samantha  
String email; sam@abc.com  
String department; M.B.B.S
```

b26t37g

```
Student std = new Student();  
std.id = 1;  
std.name = "Samantha";  
std.email = "sam@abc.com";  
std.department = "M.B.B.S";
```

Inheritance

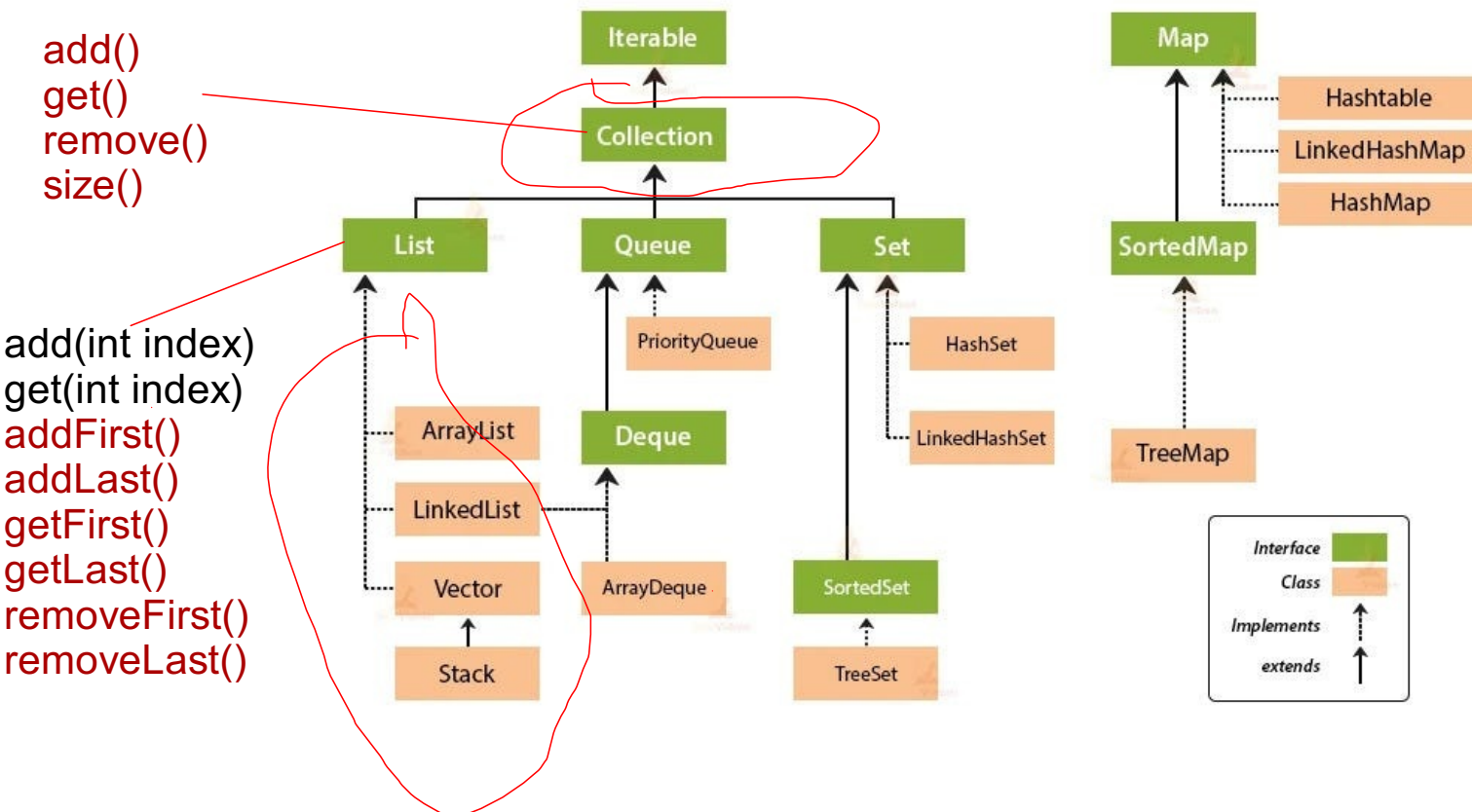
```
//parent  
class Animal{
```

```
//child  
class Lion extends Animal{  
class Hyena extends Animal{
```

```
//parent  
interface Animal {
```

```
//child  
class Lion implements Animal{  
class Hyena implements Animal{
```

Collection Framework Hierarchy in Java



Overloading

```
LinkedList numberList = new LinkedList();  
numberList.add(20);  
numberList.add("Samantha");  
numberList.add(1, "Sairam");
```

Overriding

```
//overriding object class method  
public String toString(){  
    return "ID: " + this.id + "\tName: " +  
    this.name + "\temail: " + this.email;  
}
```

Java Database Connectivity

MySQL Connector for Java

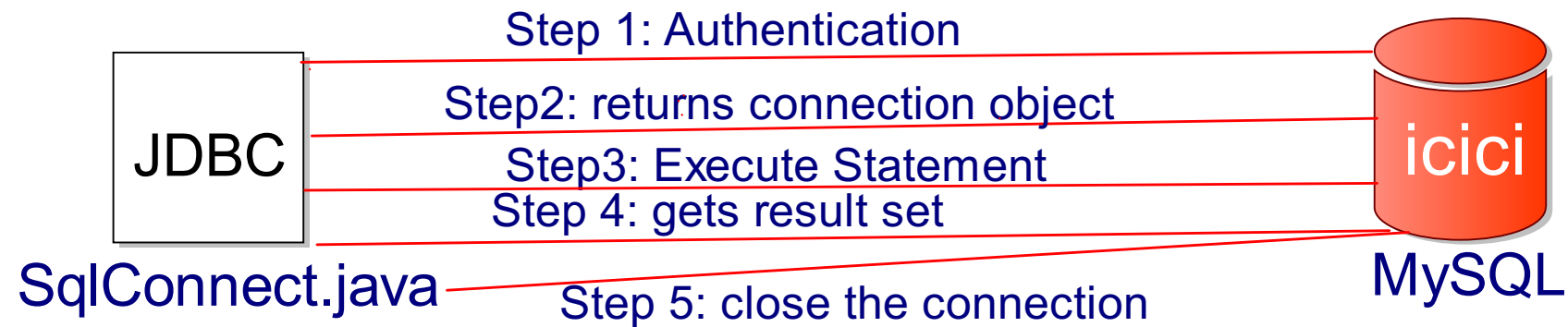
Authentication : hostname, portnumber, database-name,
username, password

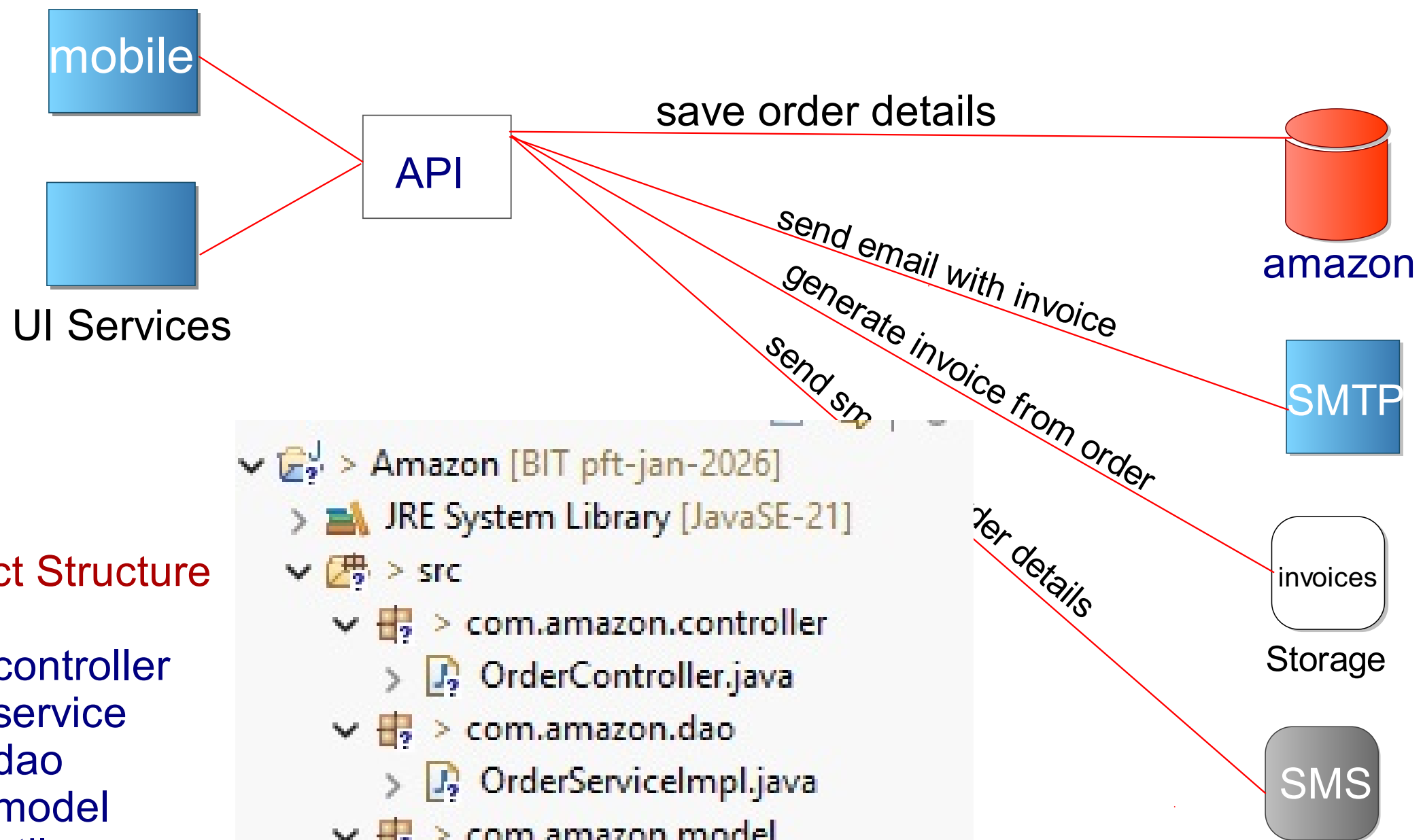
Connection object from mysql

Statements

ResultSet as response for Execute Statement

close the connection

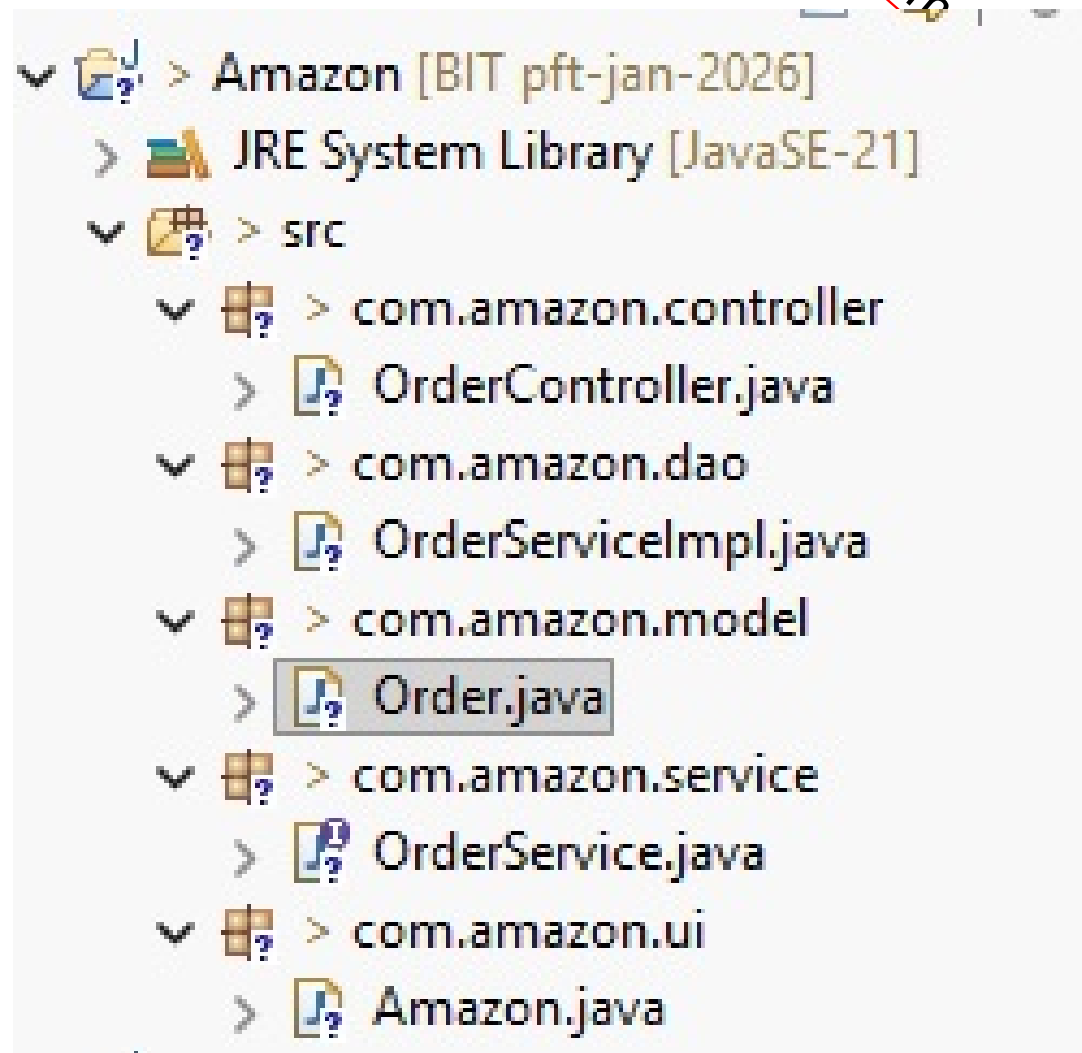




Project Structure

src
controller
service
dao
model
util

bin
web



SOLID Principles

SRP -> Single Responsibility Principle

```
public interface OrderService {  
    public Boolean saveOrder(Order order);  
}  
  
public class OrderServiceImpl implements OrderService {  
    @Override  
    public Boolean saveOrder(Order order) {  
    }  
}  
  
public interface InvoiceService {  
    public void generateOrderInvoice(Order order);  
}
```

```
public class InvoiceServiceImpl implements InvoiceService {  
    @Override  
    public void generateOrderInvoice(Order order) {  
    }  
}  
  
public interface NotificationService {  
    public void notificcation(String to, String message,  
String subject);  
}  
  
public class EmailService implements NotificationService {  
    @Override  
    public void notificcation(String to, String message,  
String subject) {  
    }  
}  
  
public class SMSService implements NotificationService {  
    @Override  
    public void notificcation(String to, String message,  
String subject) {  
    }  
}
```

Discount Calculation

Republic Sale -> 10%
Freedom Sale -> 20%
GIF Sale -> 25%
Other -> 5%
None -> 0%

```
interface DiscountStrategy{  
class DiscountStrategy implements DiscountStrategy{  
    //code for all strategies using if-else if- else  
}
```

Strategies based on condition

Open/Closed Principle

Liskov Substitution Principle

Removing unwanted inheritance
Exception Handling
Custom Exception

Interface Segregation Principle

Dependency Inversion Principle

Design Patterns

Creational -> how objects are created

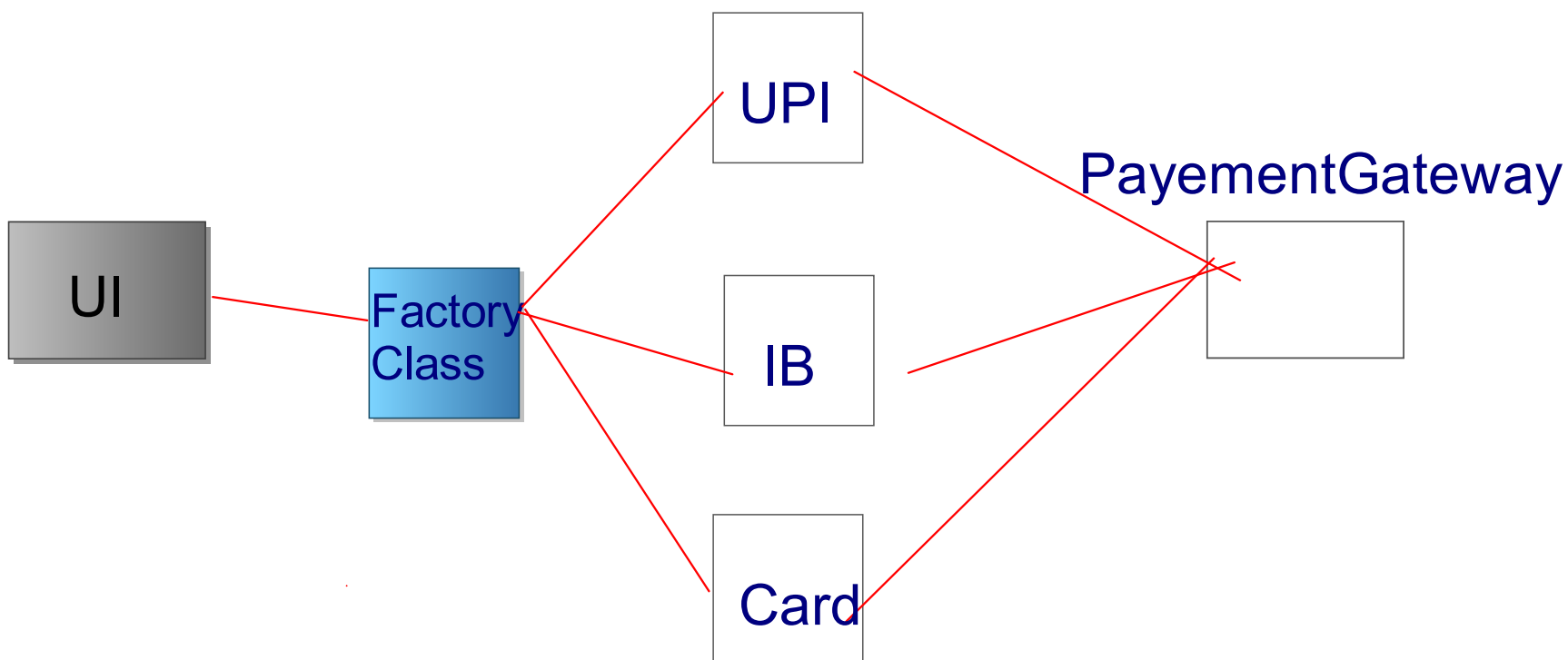
Structural -> how objects are connected

Behavioral -> how objects communicate

Singleton

transacted_on	transaction_type	name	amount
2026-01-10 14:59:17	1	Withdrawl	345.00
2026-01-10 14:59:17	1	Withdrawl	2345.00
2026-01-10 14:59:17	2	Deposit	45678.00

Factory



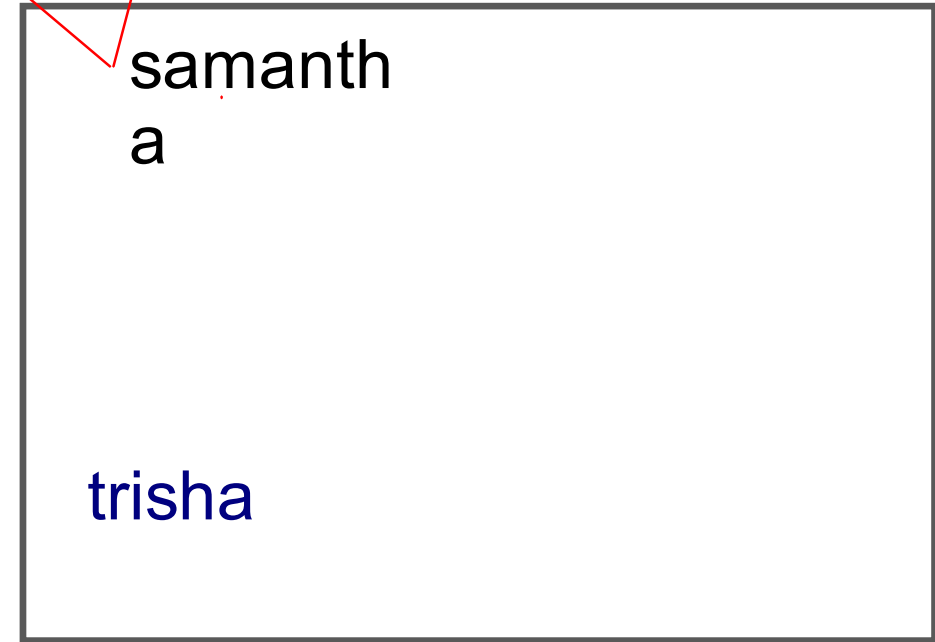
class	PascalCase	Noun	LinkedList, Payment
interface	PascalCase	Adjective	Collection, PaymentProcessor
method	camelCase	verb	next(), nextLine(), display()
variables	camelCase		name, firstName, lastName
constants	UPPER_SNAKE_CASE		HOST_NAME, PASSWORD
package	lowercase (seperated with dot)		edu.bit.training
enum	UPPERCASE		MALE, FEMALE, UPI, CARD

String name = "samantha";

String x = samantha";

String a = new String("samantha");

==



trisha

String pool